

Documentación Algoritmo Determinista de Cifrado

Ciencias de la computacion 2

Facultad de Ingeniería
Ingeniería de Sistemas

Angie Nathaly Cisneros Guerrero 20242020084
Juan Manuel Castillo Acevedo 20232020090
Luna Alejandra Sandoval Rodriguez 20241020053
Maria Juliana Alzate Saavedra 20232020082

Septiembre de 2026

Índice

1. Marco Teórico	2
1.1. Naturaleza y Propiedades de las Funciones Hash	2
1.2. Arquitectura de la Red de Feistel	2
1.3. Fundamentos Lógicos: Compuertas XOR y AND	2
2. Pseudocódigo del Algoritmo	3
3. Ejecucion del codigo	4
4. Ejemplo	5
5. Conclusiones: Sustentos Técnicos del Determinismo Algorítmico	6

1. Marco Teórico

1.1. Naturaleza y Propiedades de las Funciones Hash

Una función hash criptográfica es un modelo matemático estructurado para tomar un volumen de datos de tamaño arbitrario y comprimirlo de manera determinista en una salida alfanumérica de longitud estrictamente fija, conocida como resumen (digest) o firma digital. A diferencia de los sistemas de cifrado convencionales que están concebidos para proteger la confidencialidad de la información permitiendo su recuperación posterior mediante una clave, las funciones hash están diseñadas deliberadamente para ser operaciones unidireccionales. Esto implica que es computacionalmente inviable aplicar ingeniería inversa para recuperar el mensaje original a partir del hash resultante. Esta irreversibilidad se logra mediante la pérdida intencional de información en operaciones aritméticas de desbordamiento y compuertas lógicas destructivas. Además, una función hash robusta debe garantizar el efecto avalancha, asegurando que la modificación microscópica de un solo bit en la entrada desencadene una mutación caótica que altere al menos la mitad de los bits del resultado, destruyendo así cualquier correlación estadística que un criptoanalista pudiera intentar trazar.

1.2. Arquitectura de la Red de Feistel

La Red de Feistel representa uno de los paradigmas arquitectónicos más importantes en la historia de la criptografía moderna, sentando las bases para estándares internacionales como el DES (Data Encryption Standard). Su genialidad algorítmica radica en que permite construir cifradores por bloques reversibles utilizando funciones matemáticas internas que no necesitan ser reversibles en absoluto. El bloque de texto plano se secciona simétricamente en dos mitades dependientes: un bloque Izquierdo (L) y un bloque Derecho (R). Durante cada ciclo o ronda de procesamiento, el bloque derecho transita intacto para convertirse en la base estructural de la siguiente etapa, mientras paralelamente es introducido en una función matemática no lineal (junto con una subclave criptográfica). El resultado caótico de esta función se utiliza para mutar severamente al bloque izquierdo mediante una operación de disyunción exclusiva. Esta dinámica de alternancia cruzada asegura que la entropía se distribuya uniformemente a lo largo de todo el registro de memoria sin descartar la información original, permitiendo que el proceso de descifrado sea estructuralmente idéntico al de cifrado, requiriendo únicamente la inversión temporal del orden de las subclaves.

1.3. Fundamentos Lógicos: Compuertas XOR y AND

En el desarrollo de primitivas criptográficas a bajo nivel, el control de la información se delega a las operaciones booleanas fundamentales ejecutadas directamente por los registros del procesador. La compuerta lógica XOR (OR Exclusivo, $\oplus$) reina indiscutiblemente en el cifrado de datos debido a su propiedad de simetría perfecta y reversibilidad matemática ($A \oplus B \oplus B = A$). Actúa como un mecanismo ideal para enmascarar información: cuando un flujo de datos estructurados colisiona mediante XOR contra una secuencia pseudoaleatoria (clave), el texto original se convierte en ruido blanco incomprensible, pero conserva el cien por ciento de su integridad latente, esperando que la aplicación de la misma clave revierta el ruido a su estado semántico. Por el contrario, la compuerta AND ($\cdot$) cumple una función diametralmente opuesta al ser una operación destructiva que descarta información vital.

Si se aplica un AND bit a bit y el resultado es cero, se vuelve imposible determinar el estado de las variables originales que originaron esa salida. En el contexto de la ingeniería criptográfica, el AND no se utiliza para cifrar mensajes, sino que se emplea intensivamente como una "máscara de bits" (Bitmask) restrictiva, obligando a los registros de memoria a truncar los desbordamientos no deseados y aislar segmentos específicos de información, garantizando que los cálculos aritméticos se mantengan confinados dentro de los límites de diseño (como forzar un registro entero a comportarse estrictamente como un bloque de 8 o 16 bits).

2. Pseudocódigo del Algoritmo

A continuación, se detalla la abstracción secuencial del diseño criptográfico implementado en el código fuente, evidenciando las directivas que convierten la entrada de texto en una estructura unidireccional:

```

1 MODULO Cifrado
2
3 FUNCION procesarBloque(charL, charR, hashAcumulado_Anterior):
4     // 1. Conversion Base y Empaquetamiento
5     asciiL = valor_entero(charL)
6     asciiR = valor_entero(charR)
7     entradaOriginal = desplazar_izquierda(asciiL, 8) OR asciiR
8
9     // 2. Ejecucion de Ronda de Feistel Simplificada
10    subclave = 0x4B
11    feistel_L = asciiR
12    // Se utiliza AND 0xFF como mascara para truncar desbordamientos
13    mutacion_derecha = asciiR + subclave
14    mutacion_derecha = mutacion_derecha AND 0xFF
15    feistel_R = asciiL XOR mutacion_derecha
16
17    salidaFeistel = desplazar_izquierda(feistel_L, 8) OR feistel_R
18
19    // 3. Difusion por Rotacion Circular (16 bits)
20    // Se extraen los 5 bits superiores y se reinseran en la base
21    rotado_izq = desplazar_izquierda(salidaFeistel, 5)
22    rotado_der = desplazar_derecha_logica(salidaFeistel, 11)
23    rotado = (rotado_izq OR rotado_der) AND 0xFFFF
24
25    // 4. Bloqueo de Reversibilidad
26    bloqueIrreversible = rotado XOR entradaOriginal
27
28    // 5. Encadenamiento Secuencial con Efecto Avalancha
29    hashAcumulado_Desplazado = desplazar_izquierda(
hashAcumulado_Anterior, 3)
30    nuevoHash = hashAcumulado_Desplazado XOR bloqueIrreversible
31
32    RETORNAR nuevoHash
33
34 FIN FUNCION

```

Listing 1: Pseudocódigo de Procesamiento de Bloques Criptográficos

3. Ejecucion del codigo

metodo procesarBloque recibe tres parámetros críticos: los caracteres izquierdo y derecho para formar un bloque de 16 bits, y el estado histórico del hash para efectuar el encadenamiento acumulativo.

- `int asciiL = (int) charL; e int asciiR = (int) charR;`: Estas sentencias iniciales se encargan de despojar a los caracteres ingresados de su formato semántico de texto, obligando al sistema a capturar su valor numérico nativo acorde al estándar de codificación ASCII en memoria.
- `int entradaOriginal = (asciiL « 8) | asciiR;`: Esta instrucción constituye el fundamento para empaquetar los datos. Desplaza matemáticamente la mitad izquierda 8 posiciones binarias para abrir un vacío lógico lleno de ceros, y posteriormente utiliza la compuerta OR inclusiva para fusionar de manera impecable el bloque derecho en ese espacio vacío. Su propósito vital es conservar una "fotografía intacta del mensaje en claro antes de su alteración.
- »■ `int feistelL = asciiR;`: Aplicando estrictamente la primera ley estructural de la Red de Feistel, esta línea toma la mitad derecha de los datos ingresados y la traslada de manera directa y sin alteraciones algebraicas para que asuma la posición izquierda del siguiente ciclo.
- »■ `int feistelR = asciiL ^ ((asciiR + subclave) & 0xFF);`: Aquí reside el motor de mutación principal. Se procesa la mitad derecha aplicándole una suma modular heurística con una constante arbitraria (0x4B), y se aplica inmediatamente una máscara de restricción lógica (& 0xFF) mediante una compuerta AND, para decapitar cualquier acarreo matemático que intente desbordar la capacidad de un byte estructural. Finalmente, este ruido controlado se estrella frontalmente contra la mitad izquierda pura utilizando la compuerta XOR (^), logrando una ofuscación segura.
- »■ `int salidaFeistel = (feistelL « 8) | feistelR;`: Utilizando la misma técnica de desplazamiento binario y superposición lógica mencionada anteriormente, esta sentencia reagrupa el bloque mutado izquierdo y derecho en una única cadena transitoria de 16 bits, solidificando matemáticamente los efectos de la ronda de Feistel.
- ''■ `int rotado = ((salidaFeistel « 5) | (salidaFeistel »>11)) & 0xFFFF;`: Puesto que la máquina virtual no dispone de operadores nativos de rotación para variables recortadas a 16 bits, esta línea simula el fenómeno de difusión perfecta aislando algebraicamente los bits que desbordan por el límite superior y forzándolos a reinsertarse matemáticamente en las posiciones inferiores mediante operadores de desplazamiento y disyunción inclusiva. La máscara AND final (& 0xFFFF) depura la variable para que no contamine el resto del registro de 32 bits.
- ''■ `int bloqueIrreversible = rotado ^ entradaOriginal;`: Esta es la maniobra arquitectónica que destruye el camino de recuperación simulando la técnica Davies-Meyer. Al aplicar una compuerta XOR entre la información completamente deformada y la "fotografía inicial, el sistema crea un tapón algebraico

que impide al atacante retrotraer las operaciones sin conocer de antemano el mensaje que está intentando vulnerar.

```
''' int nuevoHash = (hashAcumulado ' 3) ^ bloqueIrreversible;; Actuando como
la directiva de encadenamiento secuencial, toma el registro histórico
del estado anterior, lo empuja deliberadamente 3 posiciones para anular
vulnerabilidades de conmutatividad geométrica (evitando que los anagramas
produzcan colisiones de firmas), y lo fusiona irreversiblemente con el
fragmento lógico recién calculado.
'''
```

»"4. Ejemplo

»"Se metoe el bloque silábico "Ho ". a las transformaciones matemáticas delineadas en el algoritmo, el estado acumulador histórico, por ser el inicio de la cadena de texto, arranca con el valor aritmético de cero absoluto.

»"Fase 1: Preparación y Empaquetado Lógico El sistema inicia ingiriendo los caracteres y extrayendo el sustrato numérico nativo: 'H' equivale a $0x48$ y 'o' corresponde a $0x6F$. La instrucción entradaOriginal toma el $0x48$, lo arrastra hacia las posiciones de alta precedencia transformándolo en $0x4800$, y le superpone el valor del segundo carácter mediante un OR lógico, consolidando el bloque combinado inicial de $0x486F$. Este estado se almacena meticulosamente para su futura ejecución final.

»"Fase 2: Colisión en la Ronda de Feistel Se procede a dividir virtualmente la cadena procesada. El carácter 'o' ($0x6F$) esquiva cualquier manipulación y cruza directamente a ocupar el hemisferio izquierdo. Simultáneamente, el sistema captura esta misma 'o' ($0x6F$), le adiciona la constante inyectada ($0x4B$), obteniendo el producto aritmético transitorio $0xBA$. Este resultado colisiona lógicamente mediante XOR contra la 'H' nativa ($0x48$), arrojando el segmento mutado $0xF2$. Al reensamblar ambos fragmentos en el registro único de 16 bits, el bloque evoluciona transformándose en el vector $0x6FF2$.

»"Fase 3: Propagación de Entropía por Rotación El vector derivado $0x6FF2$ (binario: 0110 1111 1111 0010) es sometido a un proceso de traslación espacial. La instrucción desplaza la totalidad de la cadena cinco índices hacia el lado izquierdo. Los bits superiores que amenazan con desbordar el registro de memoria son meticulosamente rescatados mediante un desplazamiento sin signo hacia el margen derecho y ensamblados nuevamente a través del operador OR, induciendo un nivel profundo de difusión posicional. Tras la aplicación de la máscara lógica delimitadora, el estado interno se asienta firmemente en el valor hexadecimal $0xFE4D$.

»"Fase 4: Feed-forward y Generación Definitiva En el paso irreversible definitivo de la arquitectura, el algoritmo confronta el subproducto encriptado caótico $0xFE4D$ directamente contra la estampa original pacífica $0x486F$. La compuerta XOR resuelve esta discrepancia aniquilando los bits coincidentes y levantando los disidentes, pariendo finalmente la criatura lógica incomprensible.

0xB622. Al ser el ciclo inaugural del proceso, este bloque no sufre alteraciones adicionales por acumulaciones históricas previas, dictaminando que el Hash Intermedio finalice con el resumen criptográfico 0x0000B622.

»”5. Conclusiones: Sustentos Técnicos del Determinismo Algorítmico

»’El análisis minucioso de la estructura algorítmica revela de manera inequívoca que este fragmento de software opera bajo un régimen matemáticamente determinista y completamente inflexible. A pesar del nivel radical de ofuscación y mutación generada a través de las operaciones aritméticas complejas, el comportamiento de los registros del procesador nunca sucumbe ante factores estocásticos externos. En ningún segmento del procesamiento de las rutinas de compilación la máquina interactúa con generadores de semillas de entropía, rutinas dependientes del ciclo del procesador, punteros de memoria inestable, o entradas caóticas del ecosistema periférico que pudieran alterar los cálculos en ejecuciones iteradas.

»’La fiabilidad y la invariabilidad del algoritmo radican enteramente en su dependencia arquitectónica de operadores a nivel bit como los desplazamientos espaciales, las compuertas exclusivas y los filtros de truncamiento AND, todos procesados bajo la precisión mecanicista estricta del hardware. Este entorno sellado y auto-contenido implica invariablemente que un bloque de caracteres de entrada específico, acoplado con su carga de estado histórico exacto, producirá sin lugar a duda el mismo hash resultante inescrutable cada vez que el microcódigo del procesador emita la orden de ejecución, cumpliendo cabalmente el requisito rector estipulado para todo sistema de validación criptográfica y protección de integridad informacional.

»’Link GitHub: <https://github.com/Thalyhx/CienciasComputacion2>